PROGRAMMING WITH PYTHON

Lekha A
Computer Applications

PES UNIVERSITY
CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Python Basics and Standard Library

Standard Library – Random, Datetime

Lekha A
Computer Applications

PROGRAMMING WITH PYTHON

Random Numbers

PES UNIVERSITY
CELEBRATING 50 YEARS

- Python has a built-in module that can be used to make random numbers.
  - random()
- Need to import the module
  - import random

PROGRAMMING WITH PYTHON

Random Numbers

PES UNIVERSITY
CELEBRATING 50 YEARS

- seed()
  - Initialize the random number generator
  - If no value is passed it uses current system time
- randrange()
  - Returns a random number between the given range

PROGRAMMING WITH PYTHON

Random Numbers

PES UNIVERSITY
CELEBRATING 50 YEARS

- randint()
  - Returns a random number between the given range
- random()
  - Returns a random float number between 0 and 1

PROGRAMMING WITH PYTHON

Random Numbers

PES UNIVERSITY
CELEBRATING 50 YEARS

- choice()
  - Returns a random element from the given sequence
- choices()
  - Returns a list with a random selection from the given sequence.
  - random.choices(population, weights=None, cum_weights=None, k=1)

PROGRAMMING WITH PYTHON

Random Numbers

PES UNIVERSITY
CELEBRATING 50 YEARS

- population: The sequence (like a list or tuple) from which to draw elements.
- weights (optional): A list of weights that correspond to the likelihood of each element being chosen.
  - Higher weights increase the probability of the element being picked.
- cum_weights (optional): Cumulative weights, an alternative to weights.
  - Each value in cum_weights is the cumulative sum of previous weights.
- k (optional): The number of elements to select. The default is 1.

PROGRAMMING WITH PYTHON

Random Numbers

PES UNIVERSITY
CELEBRATING 50 YEARS

- shuffle()
  - Takes a sequence and returns the sequence in a random order
- sample()
  - Returns a given sample of a sequence

PROGRAMMING WITH PYTHON

Random Numbers

PES UNIVERSITY
CELEBRATING 50 YEARS

random()

```
import random
```

```
random.seed(89)
```

random

```
print("Random number of float between 0 and 1: ", random.random())
```

```
Random number of float between 0 and 1:  0.08109066462388448
```

uniform

```
print("Random number of float between 10 and 20: ", random.uniform(10, 20))
```

```
Random number of float between 10 and 20:  16.068027891459295
```

randint

```
print("Random number of integer between 1 and 10: ", random.randint(1, 10))
```

```
Random number of integer between 1 and 10:  5
```

PROGRAMMING WITH PYTHON

Random Numbers

PES UNIVERSITY
CELEBRATING 50 YEARS

randrange

```
print("Random number of integer between a range of 10 to 20: ", random.randrange(10, 20))
```

```
Random number of integer between a range of 10 to 20:  12
```

```
print("Random number of integer between a range of 10 to 200 with a step value of 10: ", random.randrange(10, 200, 10))
```

```
Random number of integer between a range of 10 to 200 with a step value of 10:  120
```

choice

```
fruits=['apple','banana','cherry','durian','enterprise','fig']
print("Randomly chosen fruit: ", random.choice(fruits))
```

```
Randomly chosen fruit:  apple
```

PROGRAMMING WITH PYTHON

Random Numbers

PES UNIVERSITY
CELEBRATING 50 YEARS

- choices

```
print("Random choice of fruits: ", random.choices(fruits))
```

```
Random choice of fruits:  ['apple']
```

```
print("Multiple random choice of fruits: ", random.choices(fruits, k=3))
```

```
Multiple random choice of fruits:  ['cherry', 'fig', 'banana']
```

```
weights = [10, 5, 1, 1, 1, 1]
print("Random choice of fruits with weights: ", random.choices(fruits, weights=weights, k=5))
```

```
Random choice of fruits with weights:  ['banana', 'apple', 'durian', 'apple', 'apple']
```

PROGRAMMING WITH PYTHON

Random Numbers

PES UNIVERSITY
CELEBRATING 50 YEARS

- sample

```
print("Random sample of 2 fruits: ",random.sample(fruits, k=2))
```

```
Random sample of 2 fruits:  ['banana', 'enterprise']
```

shuffle

```
numbers=[1,2,3,4,5,6,7,8,9,10]
random.shuffle(numbers)
print("Shuffled list: ", numbers)
```

```
Shuffled list:  [3, 5, 2, 4, 7, 9, 10, 6, 1, 8]
```

PROGRAMMING WITH PYTHON

Standard Library - datetime

PES UNIVERSITY
CELEBRATING 50 YEARS

- Defining dates and times
- The datetime module primarily includes the following important classes
  - datetime Class
    - Represents a specific point in time with both date and time components (year, month, day, hour, minute, second, microsecond). Instances of this class are immutable.

PROGRAMMING WITH PYTHON

Standard Library - datetime

PES UNIVERSITY
CELEBRATING 50 YEARS

- date Class
  - Represents a date (year, month, day) without a specific time. Instances of this class are also immutable.
- time Class
  - Represents a time (hour, minute, second, microsecond) without a specific date.

PROGRAMMING WITH PYTHON

Standard Library - datetime

PES UNIVERSITY
CELEBRATING 50 YEARS

- These classes offer various methods and functionalities to perform operations like creating, formatting, manipulating, and comparing dates and times.
  - Creating date or time objects.
  - Performing arithmetic operations (addition, subtraction) on dates and times.

PROGRAMMING WITH PYTHON

Standard Library - datetime

- Formatting dates and times into strings and parsing strings into date/time objects.
- Extracting individual components (year, month, day, etc.) from date/time objects.
- Comparing and calculating differences between dates and times.

PROGRAMMING WITH PYTHON

Standard Library - datetime

PES UNIVERSITY
CELEBRATING 50 YEARS

- Defining dates and times
  - Create various objects like datetime.datetime for specific dates and times, datetime.date for just dates, datetime.time for just times, and datetime.timedelta for durations.

PROGRAMMING WITH PYTHON

Standard Library - datetime

PES UNIVERSITY
CELEBRATING 50 YEARS

- Calculations
  - Add, subtract, and compare dates, times, and durations with intuitive syntax. Calculate age, time elapsed, or future deadlines.
- Timezones
  - Manage and convert timestamps across different timezones to avoid confusion and timezone-related errors.

PROGRAMMING WITH PYTHON

Standard Library - datetime

PES UNIVERSITY
CELEBRATING 50 YEARS

- Formatting
  - Convert date and time objects to human-readable strings in various formats like "DD/MM/YYYY" or "HH:MM:SS".
- Parsing
  - Parse strings representing dates and times into corresponding objects, ensuring proper data extraction.

PROGRAMMING WITH PYTHON

Standard Library – datetime - Benefits of using datetime

PES UNIVERSITY
CELEBRATING 50 YEARS

- Standardized and reliable
  - Consistent date and time manipulation across your Python code.
- Improved code readability
  - Makes your code clearer and easier to understand, especially when dealing with time-related logic.

PROGRAMMING WITH PYTHON

Standard Library – datetime - Benefits of using datetime

PES UNIVERSITY
CELEBRATING 50 YEARS

- Reduced errors
  - Eliminates inconsistencies and errors arising from manual date and time calculations.
- Enhanced functionality
  - Unlocks advanced features like timezone handling and internationalization.

PROGRAMMING WITH PYTHON

Standard Library – datetime - Examples

- ## datetime

```
from datetime import datetime, date, time, timedelta
from datetime import date, time, datetime
```

Creating a datetime object for the current date and time

```
current_datetime = datetime.now()
print("Current Date and Time:", current_datetime)
```

```
Current Date and Time: 2025-01-24 21:06:26.823512
```

PROGRAMMING WITH PYTHON

Standard Library – datetime - Examples

PES UNIVERSITY
CELEBRATING 50 YEARS

Date arithmetic

-

```
future_date = current_datetime + timedelta(days=7)
print("Future Date:", future_date)
future_time = current_datetime + timedelta(hours=2)
print("Future Date:", future_time)
```

```
Future Date: 2025-01-31 21:06:26.823512
Future Date: 2025-01-24 23:06:26.823512
```

Parameters for timedelta

days, seconds, microseconds, milliseconds, minutes, hours, weeks

PROGRAMMING WITH PYTHON

Standard Library – datetime - Examples

PES UNIVERSITY
CELEBRATING 50 YEARS

- Formatting and Parsing

```
formatted_date = current_datetime.strftime('%Y-%m-%d %H:%M:%S')
print("Formatted Date:", formatted_date)
```

```
Formatted Date: 2025-01-24 21:06:26
```

```
parsed_date = datetime.strptime(formatted_date, '%Y-%m-%d %H:%M:%S')
print("Parsed Date:", parsed_date)
```

```
Parsed Date: 2025-01-24 21:06:26
```

Timezones and Localization

```
import pytz
```

```
pytz.all_timezones
```

PROGRAMMING WITH PYTHON

Standard Library – datetime - Examples

PES UNIVERSITY
CELEBRATING 50 YEARS

-
```
utc_now = datetime.now(pytz.utc)
local_time = utc_now.astimezone(pytz.timezone('Asia/Dubai'))
formatted_time = local_time.strftime('%Y-%m-%d %H:%M:%S %Z')
print("Formatted Time with Time Zone:", formatted_time)
```

```
Formatted Time with Time Zone: 2025-01-24 19:36:43 +04
```

PROGRAMMING WITH PYTHON

Standard Library – datetime - Examples

PES UNIVERSITY
CELEBRATING 50 YEARS

- Creating a timezone-aware datetime object

```
eastern = pytz.timezone('US/Eastern')
localized_time = eastern.localize(datetime(2025, 1, 31, 12, 0, 0))
print("Localized Time:", localized_time)
```

```
Localized Time: 2025-01-31 12:00:00-05:00
```

PROGRAMMING WITH PYTHON

Standard Library – datetime - Examples

PES UNIVERSITY
CELEBRATING 50 YEARS

- ### Represents dates without times

```
current_date = date.today()
print("Current Date:", current_date)
```

```
Current Date: 2025-01-24
```

Represents times without dates

```
current_time = time(hour=9, minute=30, second=0)
print("Current Time:", current_time)
```

```
Current Time: 09:30:00
```

Combines a date and a time into a datetime object

```
combined_datetime = datetime.combine(current_date, current_time)
print("Combined Datetime:", combined_datetime)
```

```
Combined Datetime: 2025-01-24 09:30:00
```

PROGRAMMING WITH PYTHON

Recap

- random
  - Seed
  - Random
  - Randint
  - Uniform
  - sample
- Benefits of datetime
- datetime
  - Defining dates and times
  - Calculations
  - Timezones
  - Formatting
  - Parsing
- Benefits of datetime

PES UNIVERSITY

CELEBRATING 50 YEARS

THANK YOU

Lekha A

Computer Applications